AI Edge (Computing)

7. Extra model info

hogeschool vives

1

Content

- **Anomaly detection**
  - Introduction
  - Unsupervised learning
  - K-means Clustering
  - Dimensionality Reduction
  - Autoencoders
- **Image classificiation**
  - Convolution & filters
  - Pooling
  - Data augmentation
  - Transfer learning
- **Object detection**
  - Process
  - Metrics/performance

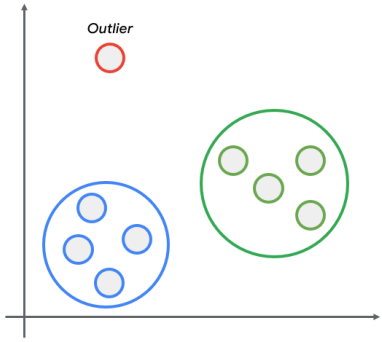
hogeschool vives

2

Anomaly detection

- **Focus on data we have <u>not seen before</u>**
- **Problem solving by using <u>unsupervised learning</u>**
  - K-means or K-NN
  - Autoencoder (NN-based)

- In **data analysis**, **anomaly detection** is the **identification of rare items**, events or observations which **raise suspicions** because they **differ significantly** from the **majority of the data**.



Extra

3

hogeschool vives

3

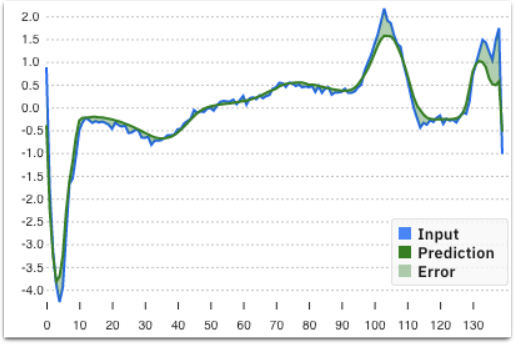
Anomaly detection

- **Three fundamental aspects of Anomaly detection**
  - Input
  - Prediction
  - Error

- **Example:**

Health

ECG Sensor



Extra

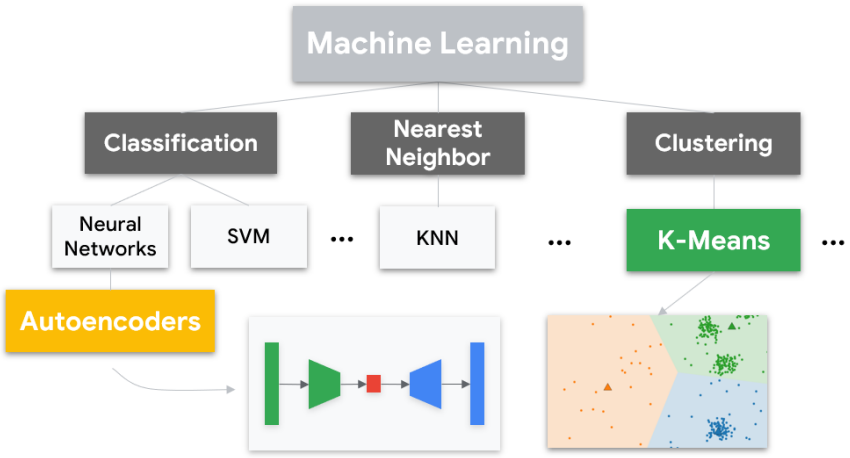
4

hogeschool vives

4

Anomaly detection

It's not all Deep Learning



Extra

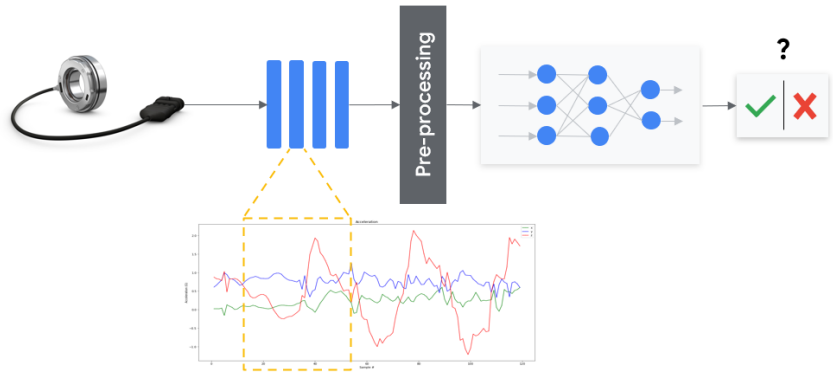
5

hogeschool vives

5

Anomaly detection

- **Example: bearing fault detection with accelerometer**



Extra

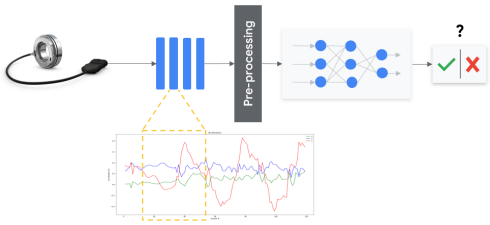
6

hogeschool vives

6

Anomaly detection

- **Example: bearing fault detection with accelerometer**
- **Challenges:**
  - Latency
  - Bandwidth (no streaming of sensor data)
  - Cost
  - Power
  - Accuracy



Extra

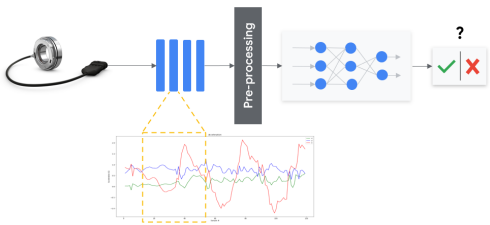
7

hogeschool vives

7

Anomaly detection

- **Example: bearing fault detection with accelerometer**
- **When doing Anomaly detection**
- **False Negative = Catastrophic failure**
  - Fault is not detected!
- **False Positive = Cost impact**
  - Error/alarm when nothing is wrong
  - Down times



Extra

8

hogeschool vives

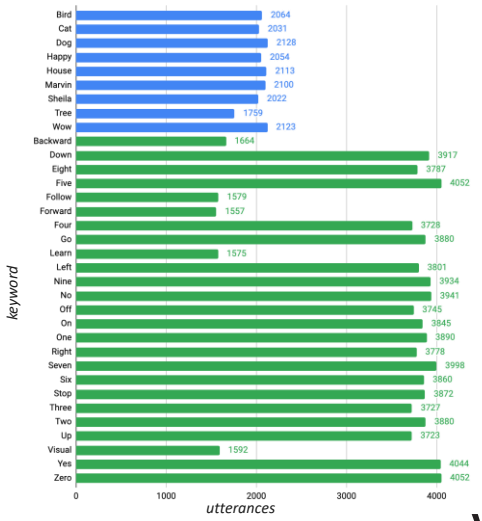
8

Anomaly detection – example datasets

- Speech commands dataset

- 25 “IoT keywords”
- 10 “unknown words”

→ We know what words we want to detect!
→ This is not an anomaly dataset!



Extra

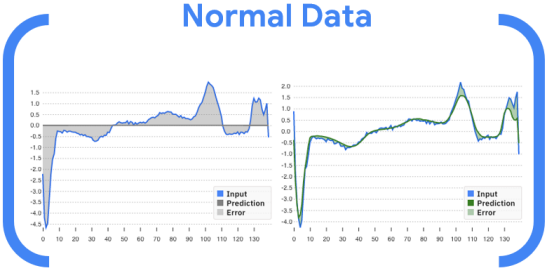
9

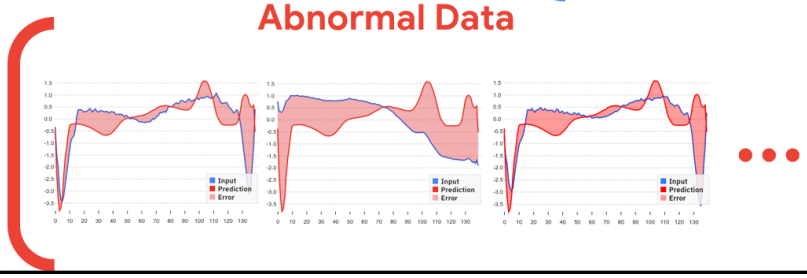
hogeschool vives

9

Anomaly detection – example datasets

- Normal data is a closed set, meaning we know what normal operation looks like and can easily capture that data.
- Abnormal data is an open set since a system can fail in any number of unpredictable ways. (Unknown unknowns)





Extra

10

hogeschool vives

10

Anomaly detection – example datasets

Unbalanced Data



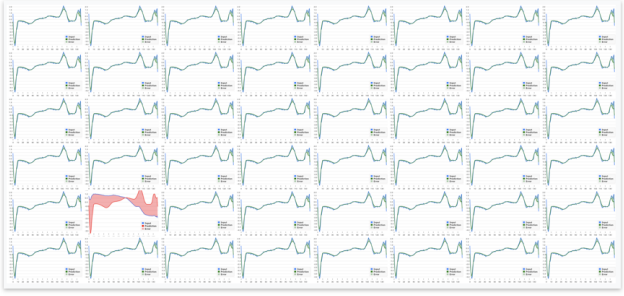
Extra

11

Anomaly detection – example datasets

- **Training on anomaly data will be very specific to the training set and therefore difficult to generalize to other deployment environments**

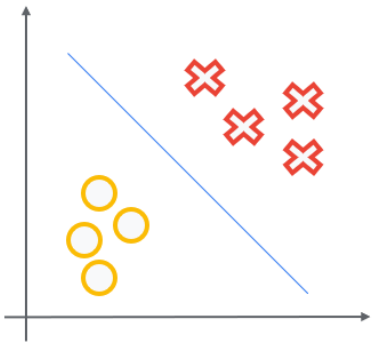
Unbalanced Data



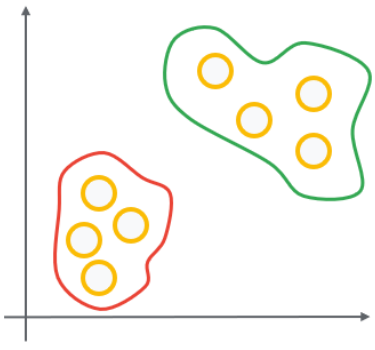
Extra

12

Anomaly detection



Supervised Learning



Unsupervised Learning

Extra

13

hogeschool vives

13

Anomaly detection - Questions

Which of the following are a use case for anomaly detection? (Select all that apply)

A. Industrial machinery monitoring

B. Face detection

C. Health monitoring

D. Email spam detection

Extra

14

hogeschool vives

14

Anomaly detection - Questions

Collecting data to train an anomaly detection system can be difficult because the data is often unbalanced and can be expensive or dangerous to capture.

A. True

B. False

Extra

15

hogeschool vives

15

Anomaly detection - Questions

Only neural networks can be used for anomaly detection.

A. True

B. False

Extra

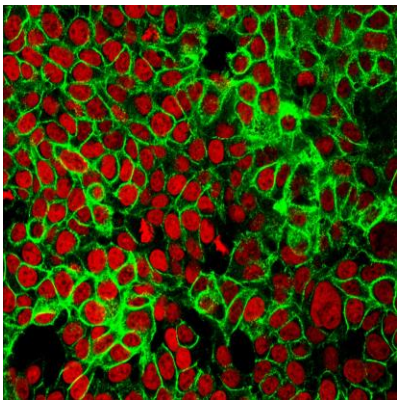
16

hogeschool vives

16

Anomaly detection – Unsupervised Learning

- **No labels!**
- **Model automatically discovers patterns in the data**
- **Uses**
  - Segmentation
  - Clustering
  - Dimensionality reduction
  - Anomaly detection



Extra

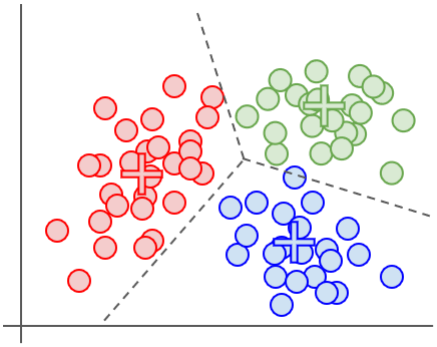
17

hogeschool vives

17

Anomaly detection – K-mean Clustering

1. Define k (e.g. k=3)
2. Randomly choose centroid for each cluster
3. Assign every sample to nearest centroid based on Euclidean distance
4. Re-compute the centroid of the cluster
5. Repeat steps 3-4
6. …until one of:
   a. Sum of distances between data points and corresponding centroid is minimized
   b. No change in centroids
   c. Maximum iterations reached



Extra

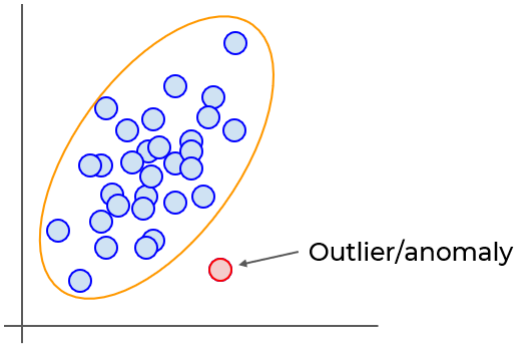
18

hogeschool vives

18

Anomaly detection

- **Examples**
  - Email spam
  - Credit card fraud
  - Motion alarm
  - Fault detection



Extra

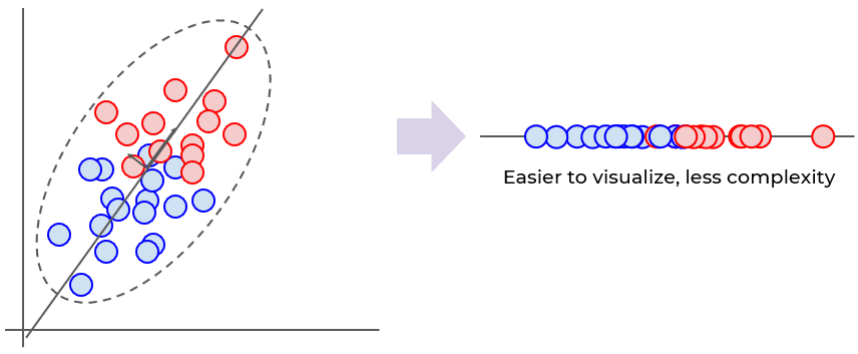
19

hogeschool vives

19

Anomaly detection – Dimensionality Reduction

- **Example: principal component analysis (PCA)**



Extra

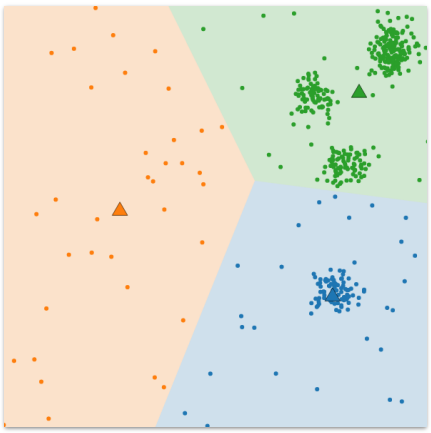
20

hogeschool vives

20

Anomaly detection – Failures

- K-means clustering can fail!
- Wrong number of k chosen →
- Scales poorly with large numbers of observations
- "Curse of dimensionality"



Extra

21

hogeschool vives

21

Anomaly detection – Autoencoders

- You don't know what you don't know.
- But you do know what you know.

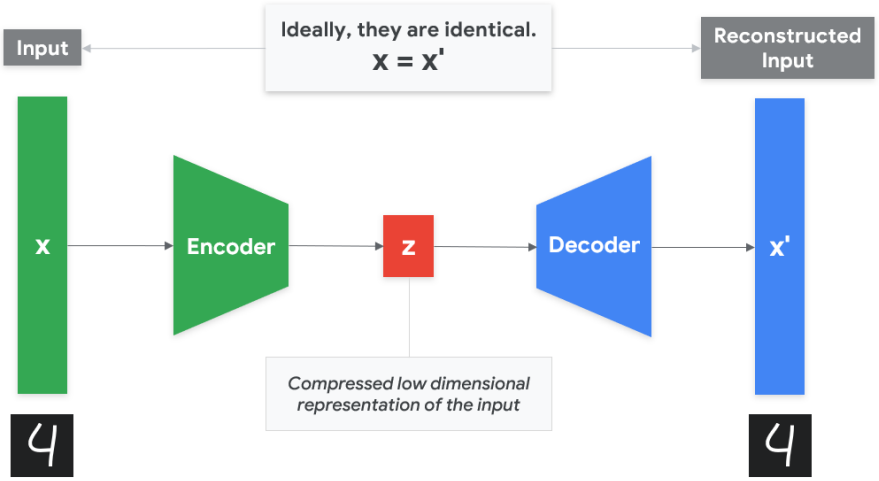
Extra

22

hogeschool vives

22

Anomaly detection – Autoencoders



Extra

23

23

Anomaly detection – Autoencoders

- **Compression of the data to bare minimum**
  - = latent space representation
- **the hidden layers of the network learn filters that are robust and <u>even capable of denoising the input data</u>**
- **During training: MSE as measure (reconstruction loss)**
  - Input image vs reconstructed image

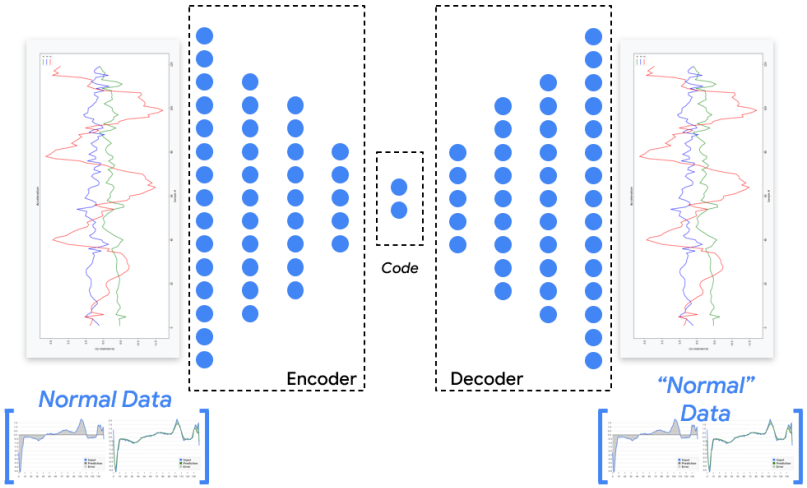
→ Perfect for anomaly detection!

Extra

24

24

Anomaly detection – Autoencoders



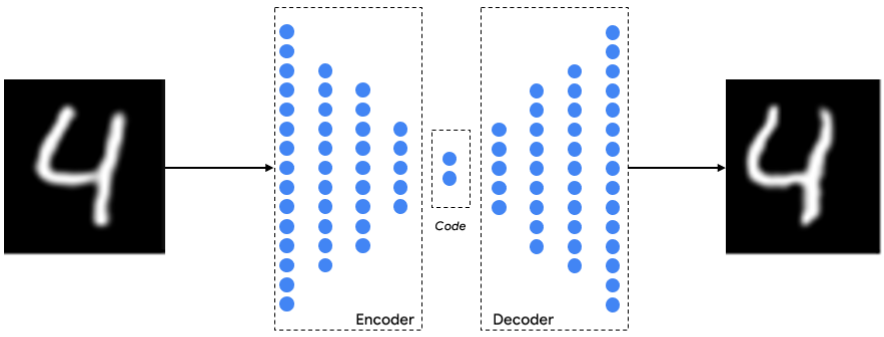
Extra

25

hogeschool vives

25

Anomaly detection – Autoencoders



Low Reconstruction Loss
(Low Mean Square Error)

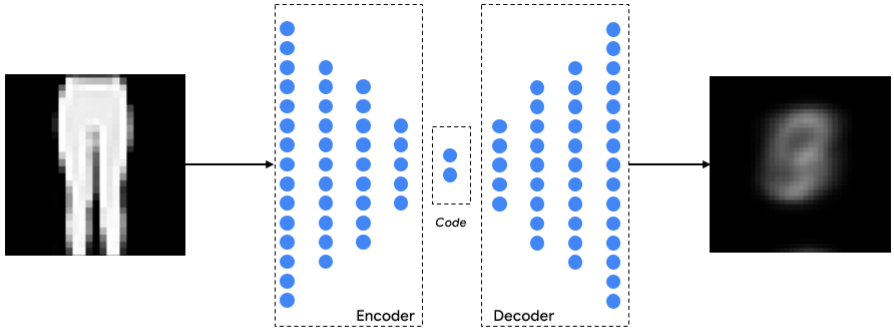
Extra

26

hogeschool vives

26

Anomaly detection – Autoencoders



Extra

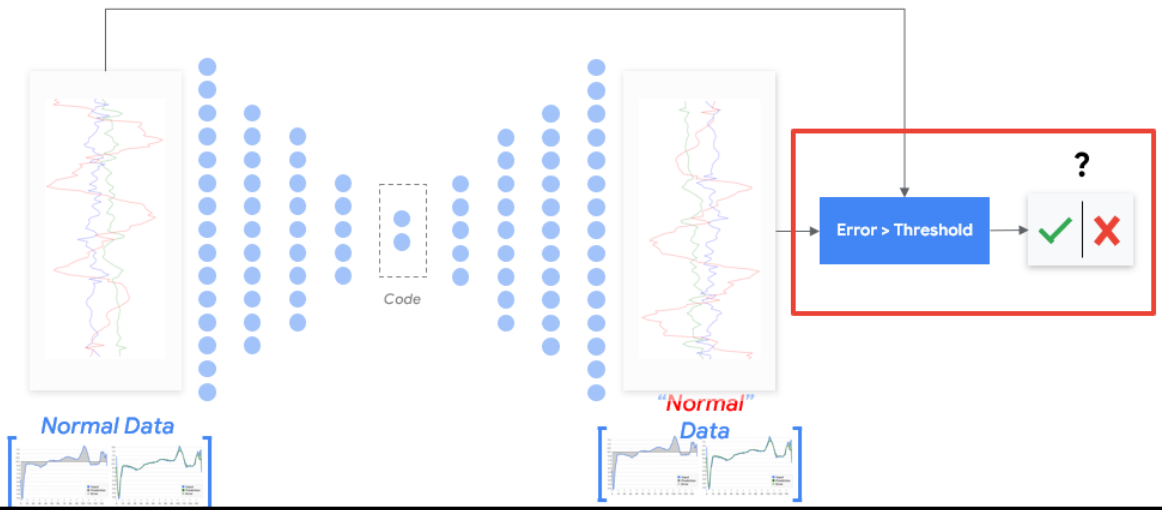
27

Low Reconstruction Loss
(Low Mean Square Error)

hogeschool vives

27

Anomaly detection – Autoencoders



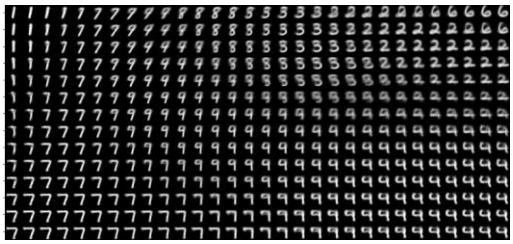
Extra

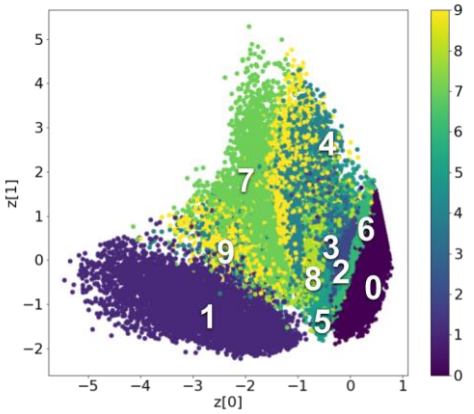
28

28

Anomaly detection – Autoencoders

- **Different types of autoencoders:**
  - Fully connected autoencoder
  - Convolutional autoencoder
  - LSTM autoencoder
  - Variational autoencoder
  - …





Extra

29

29

Anomaly detection – Autoencoders

- **Challenges with autoencoders:**
  - Size of latent space?
  - Be sure that the training data has no anomaly data!

Extra

30

30

Anomaly detection – Questions

Autoencoders require both normal and anomalous data to train.

A. True

B. False

Extra

31

hogeschool vives

31

Anomaly detection – Questions

Latent space is a compact, lower-dimensional representation of similar data points that are close together in space.

A. True

B. False

Extra

32

hogeschool vives

32

Anomaly detection – Questions

Which of the following is a disadvantage of k-means clustering?

A. It only works with data that has a gaussian distribution

B. It does not support more than 5 clusters

C. It does not scale well with very high dimensional data

D. It requires a large neural network

Extra

33

hogeschool vives

33

Image Classification

- **Remember: Image classification vs object detection**
- **Digital images - basics**

Extra

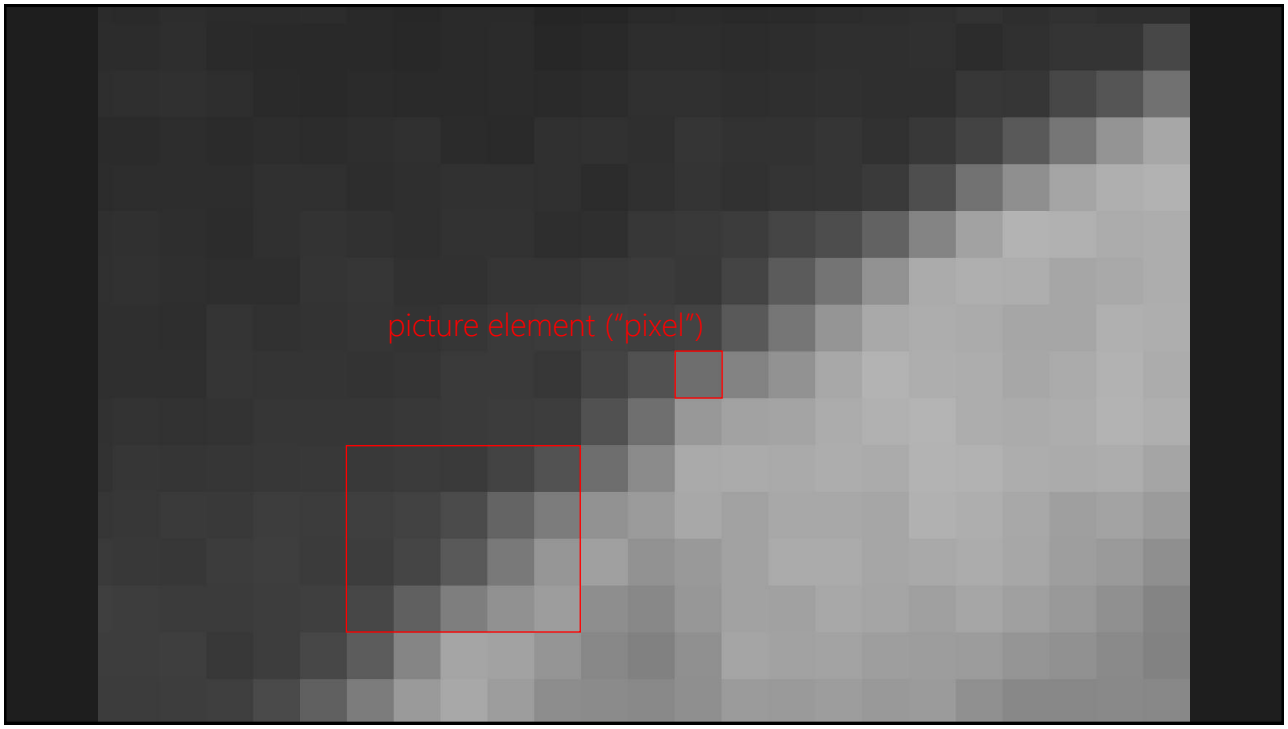
34

hogeschool vives

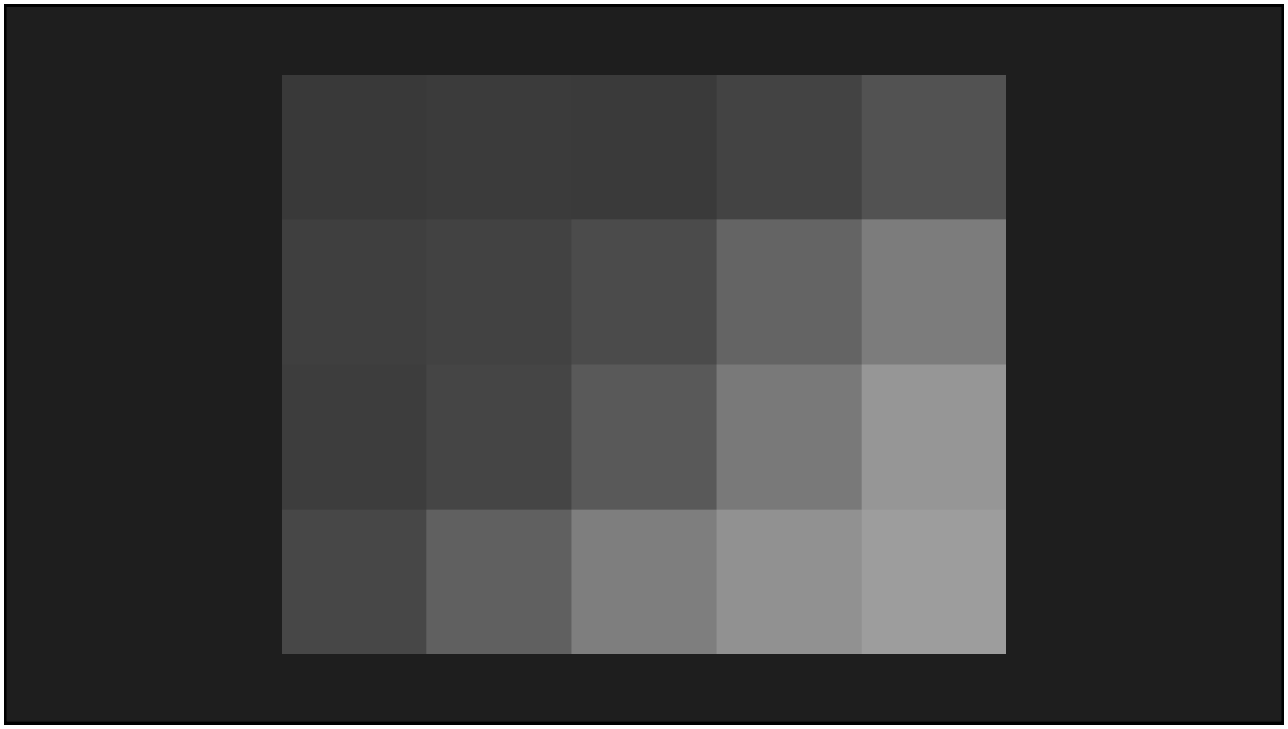
34



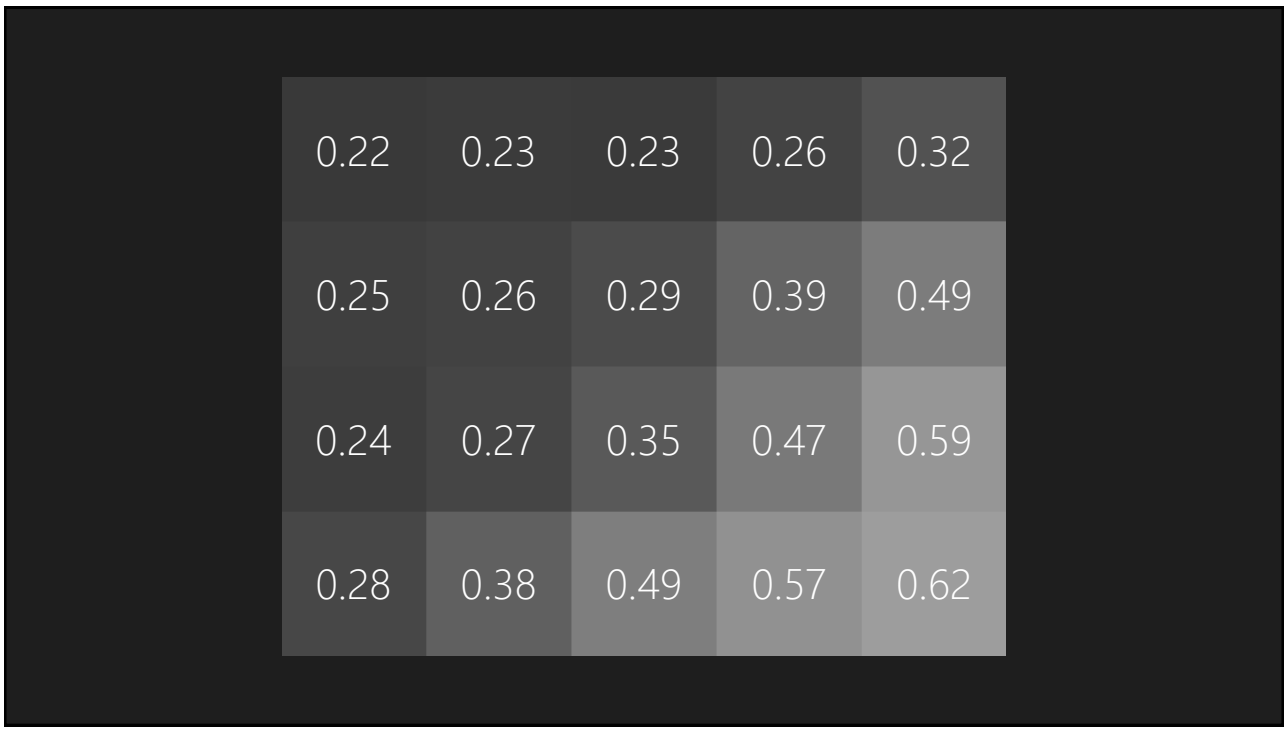
35



36



37

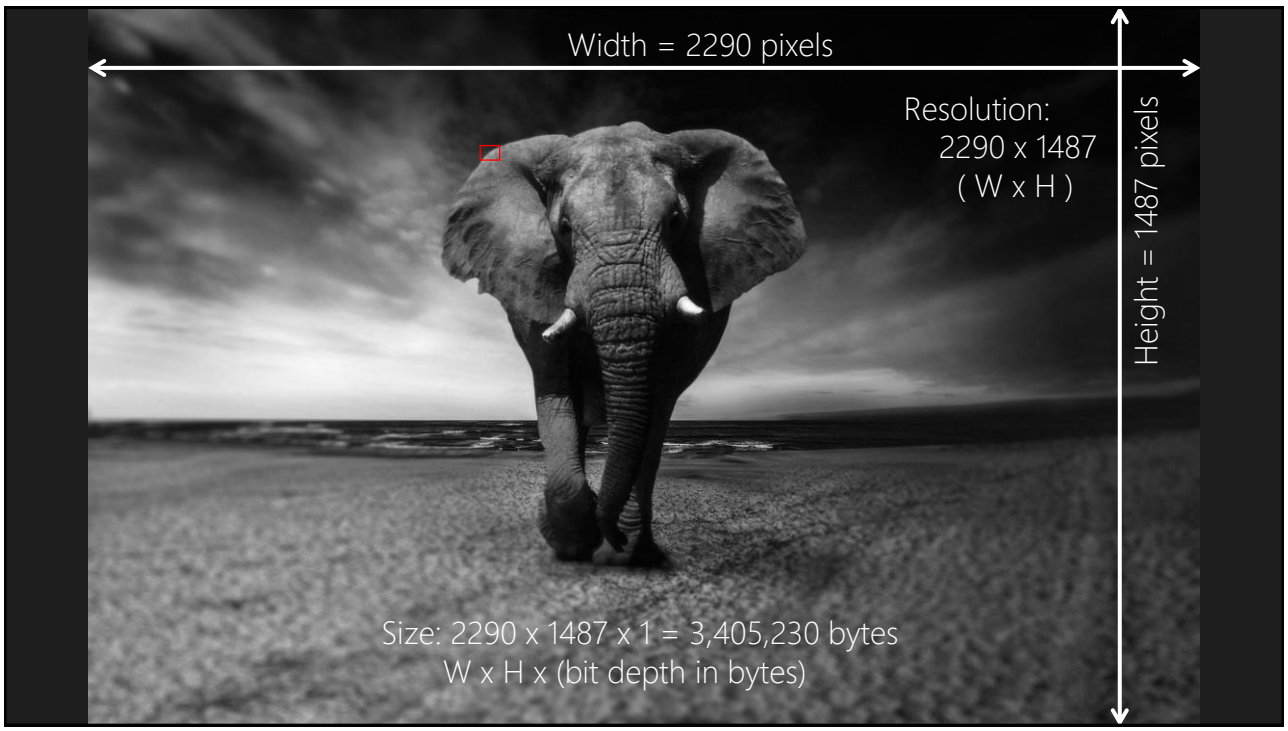

0.22 0.23 0.23 0.26 0.32
0.25 0.26 0.29 0.39 0.49
0.24 0.27 0.35 0.47 0.59
0.28 0.38 0.49 0.57 0.62

38

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Bit depth: 8 bits
- 0 = black
- 255 = white

39



40



41



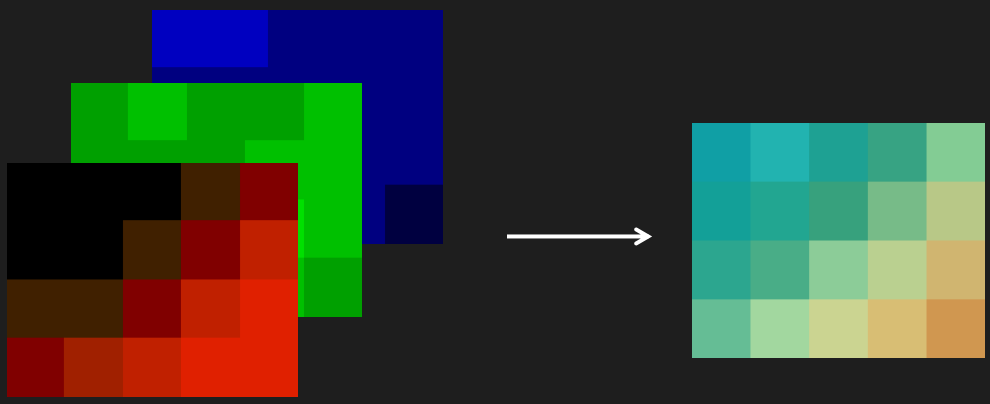
42

R: 16 G: 159 B: 165	R: 34 G: 179 B: 176	R: 30 G: 161 B: 147	R: 55 G: 163 B: 131	R: 131 G: 204 B: 148
R: 19 G: 160 B: 152	R: 34 G: 166 B: 145	R: 55 G: 161 B: 125	R: 119 G: 187 B: 136	R: 184 G: 200 B: 135
R: 44 G: 166 B: 143	R: 73 G: 173 B: 135	R: 140 G: 204 B: 152	R: 186 G: 208 B: 144	R: 208 G: 181 B: 112
R: 101 G: 189 B: 149	R: 162 G: 215 B: 159	R: 203 G: 212 B: 145	R: 216 G: 190 B: 116	R: 208 G: 151 B: 80

Bit depth: 24 bits
- 1 byte: Red
- 1 byte: Green
- 1 byte: Blue

Optional:
- 1 byte: Alpha

43



44

Image Classification

Classification types

- **Binary classification**

One-vs-One



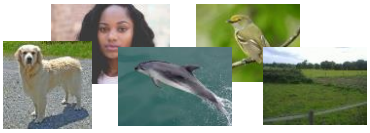
vs.



One-vs-Rest



vs.



Extra

45

Image Classification

Classification types

- **Multiclass classification**



vs.



vs.



vs.



vs.



vs.



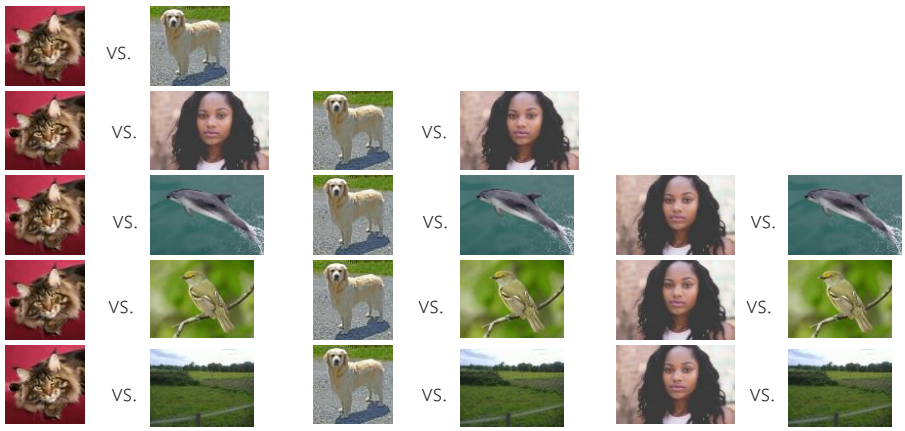
Extra

46

Image Classification

Classification types

- **Multiclass classification: one vs one**



Extra

47

hogeschool vives

47

Image Classification

Classification types

- **Multiclass classification: one vs rest**



Extra

48

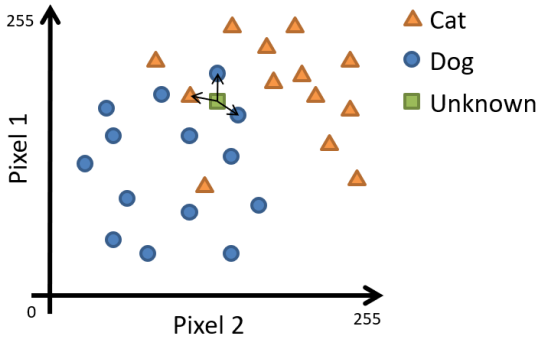
hogeschool vives

48

Image Classification

How do we do this?

- **k-Nearest Neighbors (k-NN)**



Extra

49

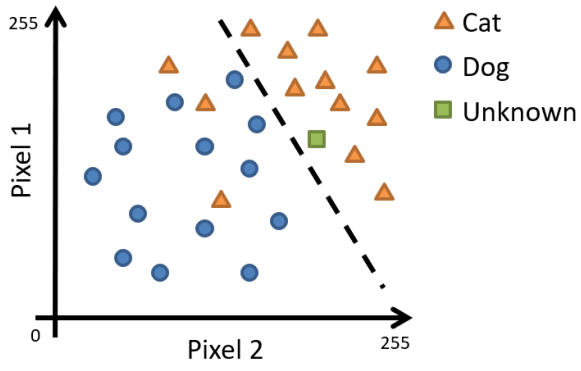
hogeschool vives

49

Image Classification

How do we do this?

- **Support Vector Machine (SVM)**



Extra

50

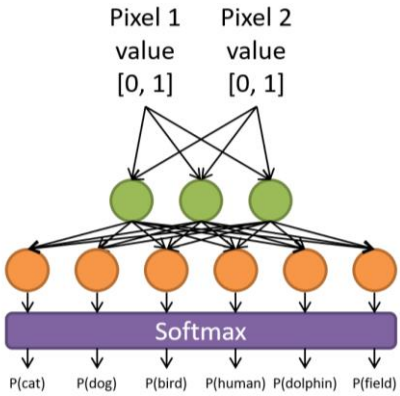
hogeschool vives

50

Image Classification

How do we do this?

- Neural Network



P(cat) + P(dog) + P(bird) + P(human) + P(dolphin) + P(field) = 1.0

Extra

51

hogeschool vives

51

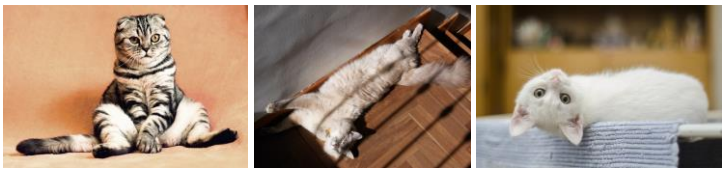
Image Classification - Challenges

How do we do this?

- Illumination



- Deformation



- Occlusion



Extra

52

hogeschool vives

52

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Kernel

0	-1	0
-1	5	-1
0	-1	0

(57 · 0) + (59 · -1) + (58 · 0) +
(63 · -1) + (66 · 5) + (75 · -1) +
(61 · 0) + (69 · -1) + (89 · 0) = 64

Output

64		

Extra

53

hogeschool vives

53

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Kernel

0	-1	0
-1	5	-1
0	-1	0

(59 · 0) + (58 · -1) + (67 · 0) +
(66 · -1) + (75 · 5) + (100 · -1) +
(69 · 0) + (89 · -1) + (121 · 0) = 62

Output

64	62	

Extra

54

hogeschool vives

54

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Kernel

0	-1	0
-1	5	-1
0	-1	0

(58 · 0) + (67 · -1) + (82 · 0) +
(75 · -1) + (100 · 5) + (124 · -1) +
(89 · 0) + (121 · -1) + (150 · 0) = 113

Output

64	62	113

Extra

55

hogeschool vives

55

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Kernel

0	-1	0
-1	5	-1
0	-1	0

(63 · 0) + (66 · -1) + (75 · 0) +
(61 · -1) + (69 · 5) + (89 · -1) +
(71 · 0) + (96 · -1) + (126 · 0) = 33

Output

64	62	113
33		

Extra

56

hogeschool vives

56

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Kernel

0	-1	0
-1	5	-1
0	-1	0

(66 · 0) + (75 · -1) + (100 · 0) +
(69 · -1) + (89 · 5) + (121 · -1) +
(96 · 0) + (126 · -1) + (145 · 0) = 54

Output

64	62	113
33	54	

Extra

57

hogeschool vives

57

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

stride = 1

Kernel

0	-1	0
-1	5	-1
0	-1	0

(75 · 0) + (100 · -1) + (124 · 0) +
(89 · -1) + (121 · 5) + (150 · -1) +
(126 · 0) + (145 · -1) + (157 · 0) = 121

Output

64	62	113
33	54	121

Extra

58

hogeschool vives

58

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

stride = 2

Kernel

0	-1	0
-1	5	-1
0	-1	0

(57 · 0) + (59 · -1) + (58 · 0) +
(63 · -1) + (66 · 5) + (75 · -1) +
(61 · 0) + (69 · -1) + (89 · 0) = 64

Output

64	

Extra

59

hogeschool vives

59

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

stride = 2

Kernel

0	-1	0
-1	5	-1
0	-1	0

(58 · 0) + (67 · -1) + (82 · 0) +
(75 · -1) + (100 · 5) + (124 · -1) +
(89 · 0) + (121 · -1) + (150 · 0) = 113

Output

64	113

Extra

60

hogeschool vives

60

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

8-bit pixel values must be between 0 and 255!

Kernel

1	5	1
5	5	5
1	5	1

$(57 \cdot 1) + (59 \cdot 5) + (58 \cdot 1) +$
$(63 \cdot 5) + (66 \cdot 5) + (75 \cdot 5) +$
$(61 \cdot 1) + (69 \cdot 5) + (89 \cdot 1) = 1925$

Output

255		

Extra

61

hogeschool vives

61

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

8-bit pixel values must be between 0 and 255!

Kernel

1	5	1
5	5	5
1	5	1

Output

255	255	255
255	255	255

Extra

62

hogeschool vives

62

Image Classification – Convolution & filtering

How do we do this?

Image

0.22	0.23	0.23	0.26	0.32
0.25	0.26	0.29	0.39	0.49
0.24	0.27	0.35	0.47	0.59
0.28	0.38	0.49	0.57	0.62

Floating point pixel values will likely be between 0.0 and 1.0

Kernel

1	5	1
5	5	5
1	5	1

Output

1.0	1.0	1.0
1.0	1.0	1.0

Extra

63

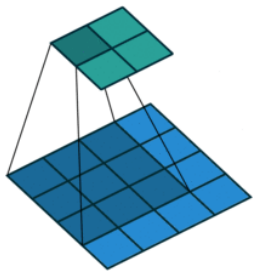
hogeschool vives

63

Image Classification – Convolution & filtering

How do we do this?

- **Valid padding**



Extra

64

hogeschool vives

64

Image Classification – Convolution & filtering

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

stride = 2

Kernel

0	-1	0
-1	5	-1
0	-1	0

Output

64	113

Extra

65

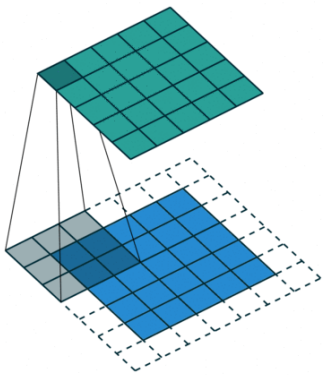
hogeschool vives

65

Image Classification – Convolution & filtering

How do we do this?

- **Same padding**



Extra

66

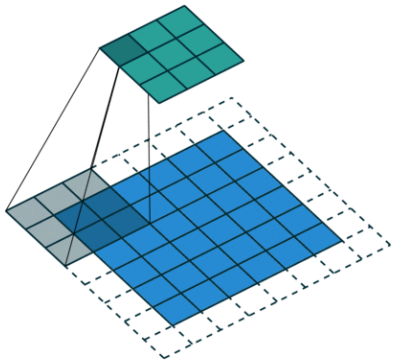
hogeschool vives

66

Image Classification – Convolution & filtering

How do we do this?

- **Padding with stride = 2**



Extra

67

Image Classification – Convolution & filtering

How do we do this?

0	0	0	0	0	0	0
0	57	59	58	67	82	0
0	63	66	75	100	124	0
0	61	69	89	121	150	0
0	71	96	126	145	157	0
0	0	0	0	0	0	0

Extra

68

Image Classification – Convolution & filtering

How do we do this?

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Kernel

0	-1	0
-1	5	-1
0	-1	0

Without padding:
- Smaller output
- Information on borders is lost

Output

64	62	113
33	54	121

Extra

69

hogeschool vives

69

Image Classification – Convolution & filtering

How do we do this?

Input Image (with padding)

0	0	0	0	0	0	0
0	57	59	58	67	82	0
0	63	66	75	100	124	0
0	61	69	89	121	150	0
0	71	96	126	145	157	0
0	0	0	0	0	0	0

stride = 1

Kernel

0	-1	0
-1	5	-1
0	-1	0

With padding:
- Larger output matrix
- Information on borders is maintained

Output

163	114	89	95	219
131	64	62	113	255
102	33	54	121	255
198	214	255	255	255

Extra

70

hogeschool vives

70

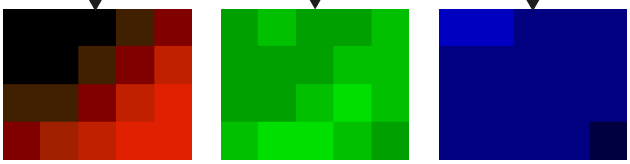
Image Classification – Convolution & filtering

How do we do this?

Original image



RGB channels



Kernel

0	-1	0
-1	5	-1
0	-1	0

0	-1	0
-1	5	-1
0	-1	0

0	-1	0
-1	5	-1
0	-1	0

Extra

71

hogeschool vives

71

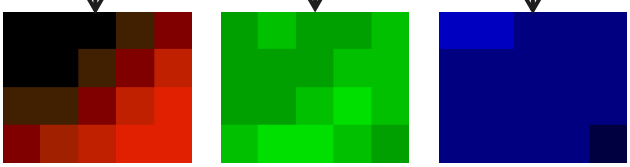
Image Classification – Convolution & filtering

How do we do this?

Original image



RGB channels



Kernel

0	-1	0
-1	5	-1
0	-1	0

-1	-2	-1
0	0	0
1	2	1

-1	-1	-1
-1	8	-1
-1	-1	-1

Extra

72

hogeschool vives

72

Image Classification – Convolution & filtering

How do we do this?

Original image (200x130)



Gaussian blur

1/16	1/8	1/16
1/8	1/4	1/8
1/16	1/8	1/16



Outline

-1	-1	-1
-1	8	-1
-1	-1	-1



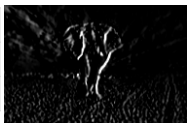
Sharpen

0	-1	0
-1	5	-1
0	-1	0



Left Sobel

1	0	-1
2	0	-2
1	0	-1



Emboss

-2	-1	0
-1	1	1
0	1	2



Top Sobel

1	2	1
0	0	0
-1	-2	-1



Extra

73

hogeschool vives

73

Image Classification – Average pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2
Find average under window:
(57 + 59 + 63 + 66) / 4 = 61.25

Output

61	

Extra

74

hogeschool vives

74

Image Classification – Average pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2
Find average under window:
(58 + 67 + 75 + 100) / 4 = 75

Output

61	75

Extra

75

hogeschool vives

75

Image Classification – Average pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2
Find average under window:
(61 + 69 + 71 + 96) / 4 = 74.25

Output

61	75
74	

Extra

76

hogeschool vives

76

Image Classification – Average pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2
Find average under window:
(89 + 121 + 126 + 145) / 4 = 120.25

Output

61	75
74	120

Extra

77

77

Image Classification – Max pooling

How do we do this?

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2

Find maximum value under window: 66

Output

66	

Extra

78

78

Image Classification – Max pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2

Find maximum value under window: 100

Output

66	100

Extra

79

hogeschool vives

79

Image Classification – Max pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2

Find maximum value under window: 96

Output

66	100
96	

Extra

80

hogeschool vives

80

Image Classification – Max pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Window: 2x2
Stride: 2

Find maximum value under window: 145

Output

66	100
96	145

Extra

81

hogeschool vives

81

Image Classification – Global pooling

How do we do this?

Image

57	59	58	67	82
63	66	75	100	124
61	69	89	121	150
71	96	126	145	157

Global average output: 91.8

92

Global max output: 157

157

Extra

82

hogeschool vives

82

Image Classification – Pooling comparison

How do we do this?

Filtered image
(198x128)



Average Pooling

(99x64)



Max Pooling

(99x64)



Extra

Image Classification – Local Translation Invariance

How do we do this?

128	255	0	0
255	128	0	0
255	0	0	0
255	0	0	0

Max Pooling

Window: 2x2
Stride: 2

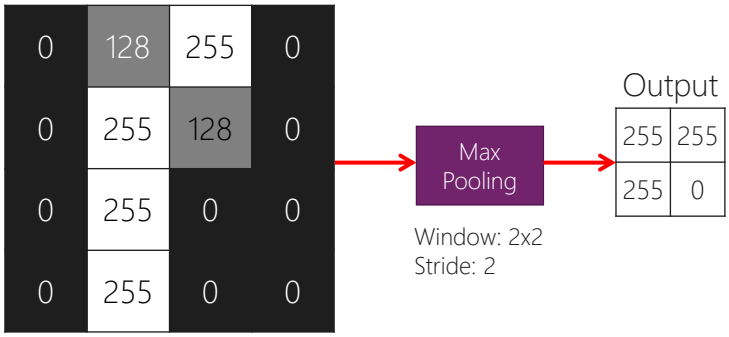
Output

255	0
255	0

Extra

Image Classification – Convolution & filtering

How do we do this?



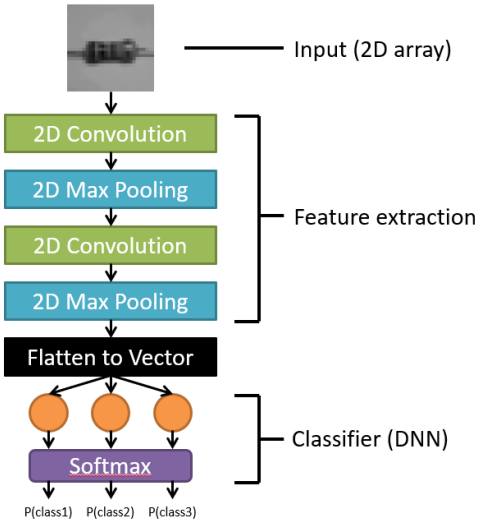
Extra

hogeschool vives

85

Image Classification – Example

Using the layers we just learned



Popular input formats:
- Float [0.0..1.0]
- Uint8 [0..255]
- Int8 [-128..127]

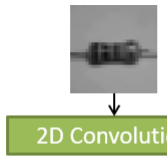
Extra

hogeschool vives

86

Image Classification – Example

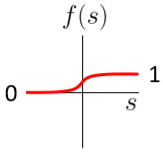
Using the layers we just learned



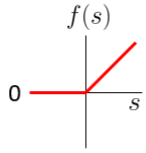
2D Convolution

Number of filters: 4
Kernel: 3x3
Stride: 1
Padding: Same

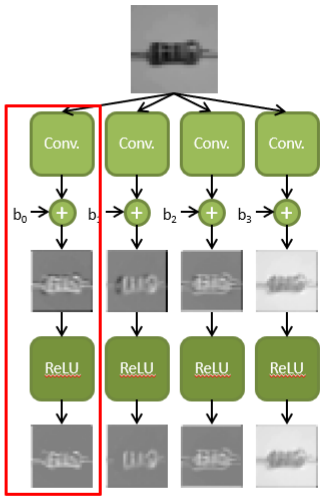
Sigmoid function



ReLU function







"neuron" or "node"

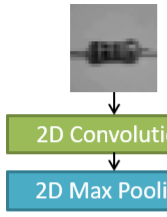
Extra

87

87

Image Classification – Example

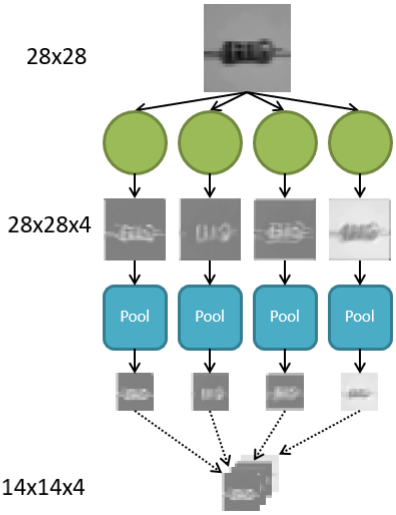
Using the layers we just learned



2D Convolution

2D Max Pooling

Shape: 2x2
Stride: 2



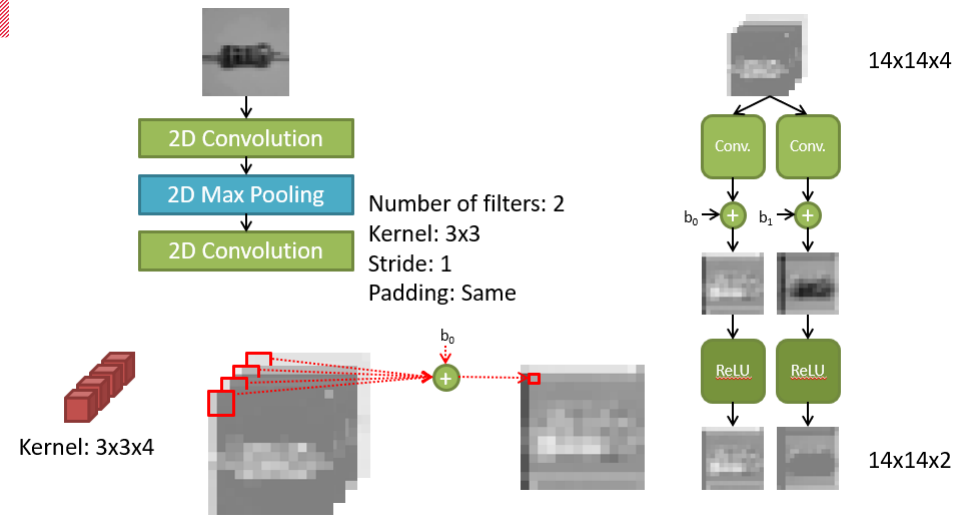
Extra

88

88

Image Classification – Example

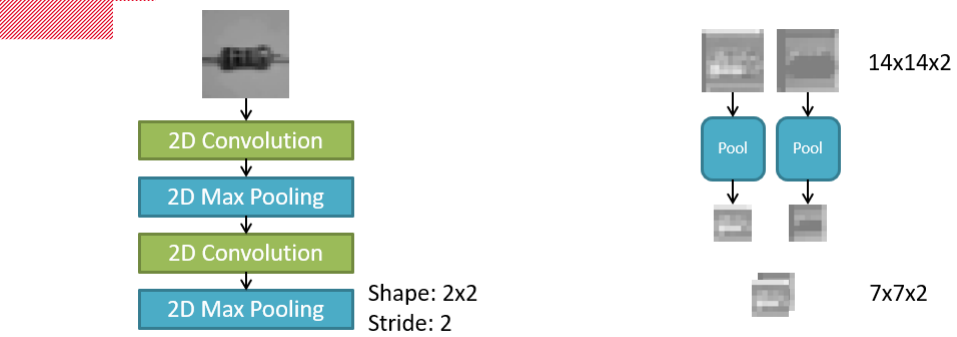
Using the layers we just learned



Extra

Image Classification – Example

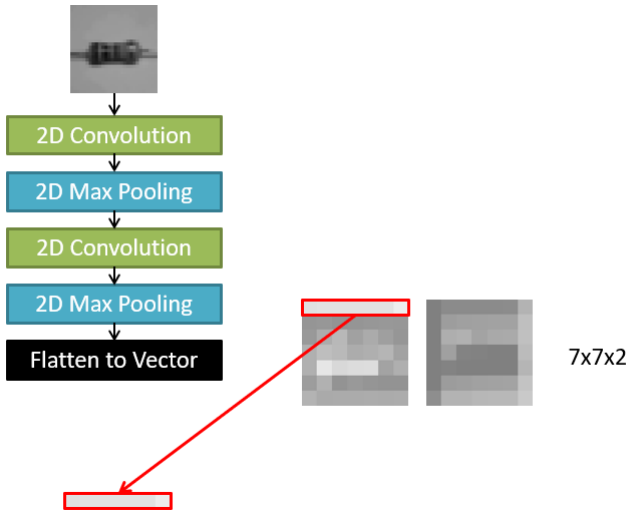
Using the layers we just learned



Extra

Image Classification – Example

Using the layers we just learned



Extra

91

hogeschool vives

91

Image Classification – Example

Using the layers we just learned



Extra

92

hogeschool vives

92

Image Classification – Questions

- Let's say you want to convolve an image (resolution of 6x4) with a 2x2 kernel using valid padding and a stride of 2. What will the resolution (w x h) of the output image be?

Extra

93

93

Image Classification – Questions

Given the following input image and kernel, solve for the value of x in the output image. The kernel is 2x2 with a stride of 1, and we're using valid padding on the input image. The output image is 3x2, and x is the second pixel to the right on the second row. Your answer should be an integer.

Image

59	58	67	82
66	75	100	124
69	89	121	150

Kernel

-1	-1
-1	3

Kernel: 2x2
Stride: 1
Padding: valid

Output

	x	

Extra

94

94

Image Classification – Questions

Given the following input image, a window size of 2x2, a stride of 2, and valid padding, what is the value of x in the output image if we are using maximum pooling? The output image is 2x1, and x is the second pixel to the right on the first row. Your answer should be an integer.

Image

59	58	67	82
66	75	100	124
69	89	121	150

Window: 2x2
Stride: 2
Padding: valid

Output

	x

Extra

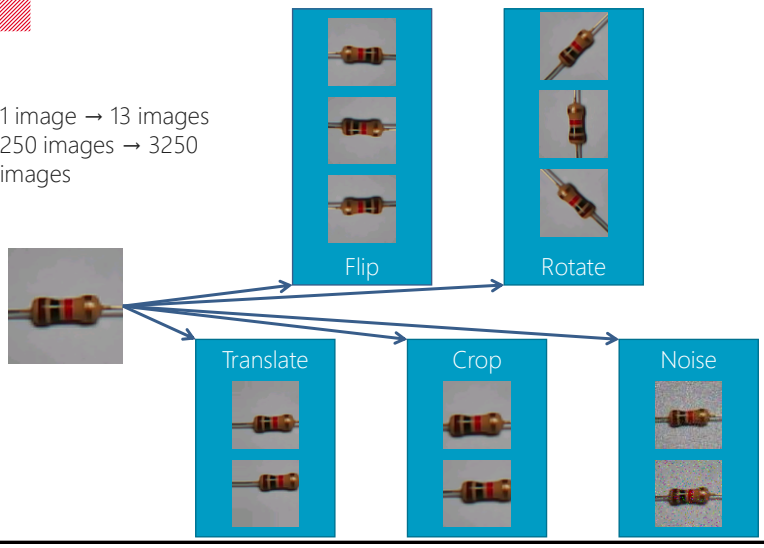
95

hogeschool vives

95

Image Classification – Data augmentation

1 image → 13 images
250 images → 3250 images



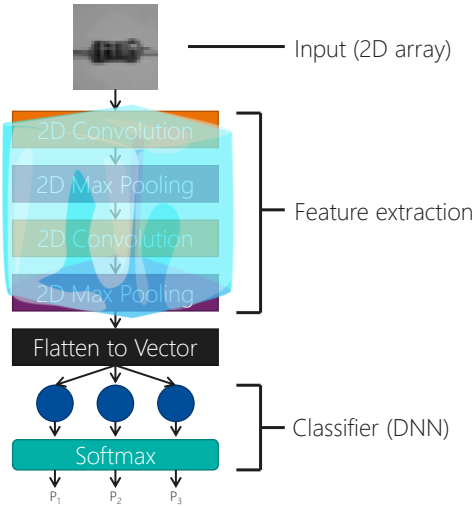
Extra

96

hogeschool vives

96

Image Classification – Transfer learning

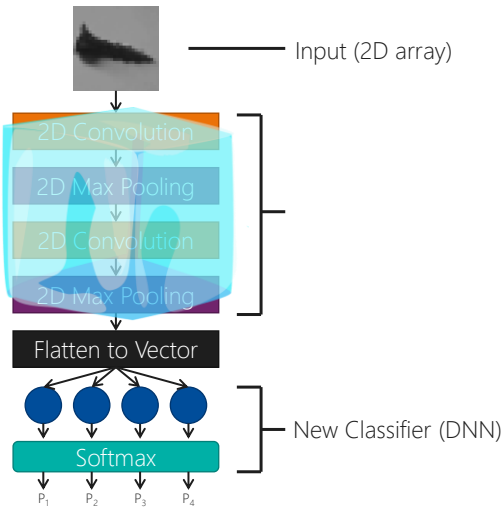


Extra

97

97

Image Classification – Transfer learning

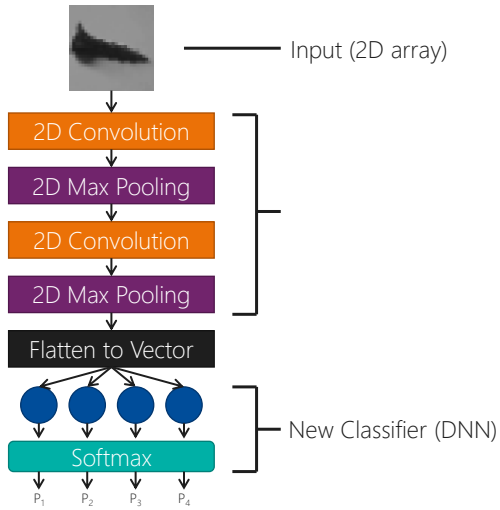


Extra

98

98

Image Classification – Fine tuning



Extra

99

99

Image Classification – Transfer learning

- **Advantages:**
  - Faster than starting from scratch
  - Smaller training dataset required

Extra

100

100

Object detection



Background	Capacitor	Diode	LED	Resistor
0.0	0.04	0.0	0.28	0.68



Background	Capacitor	Diode	LED	Resistor
0.0	0.77	0.0	0.0	0.23

Extra

101

hogeschool vives

101

Object detection - definitions

- **Image Classification**
  - Predict class of object in an image
- **Object Localization**
  - Locate presence of object(s) in an image
- **Object Detection**
  - Locate and classify object(s) in an image

Extra

102

hogeschool vives

102

Object detection – Process of detection



Extra

103

Object detection – Process of detection



Extra

104

Object detection – Process of detection

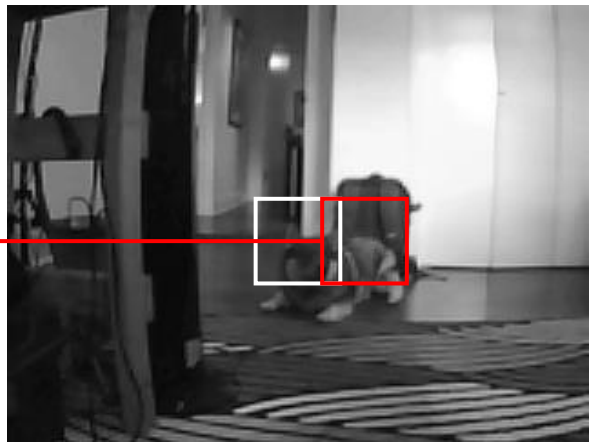


Extra

105

105

Object detection – Process of detection



Extra

106

106

Object detection – Process of detection



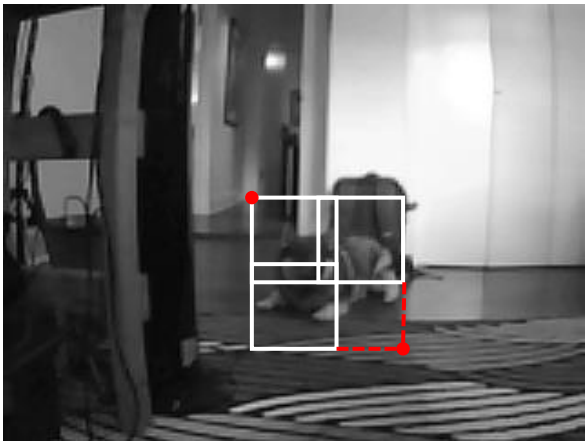
Extra

107

hogeschool vives

107

Object detection – Process of detection



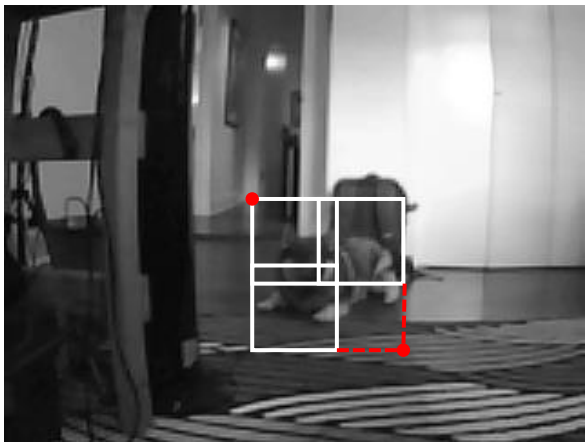
Extra

108

hogeschool vives

108

Object detection – Process of detection



Extra

109

Object detection – Performance

Confusion Matrix

Actual Label \ Predicted Label	Ball	Dog	Toy
Ball	205	10	1
Dog	6	199	0
Toy	9	17	223

Precision, Positive Predictive Value (PPV):
What proportion of positive predictions was actually correct?

$$Precision = \frac{TP}{TP + FP} = \frac{\square}{\square + \square}$$

$$= \frac{199}{199 + 27} = 0.881$$

Extra

110

Object detection – Performance

Confusion Matrix

Actual Label \ Predicted Label	Ball	Dog	Toy
Ball	205	10	1
Dog	6	199	0
Toy	9	17	223

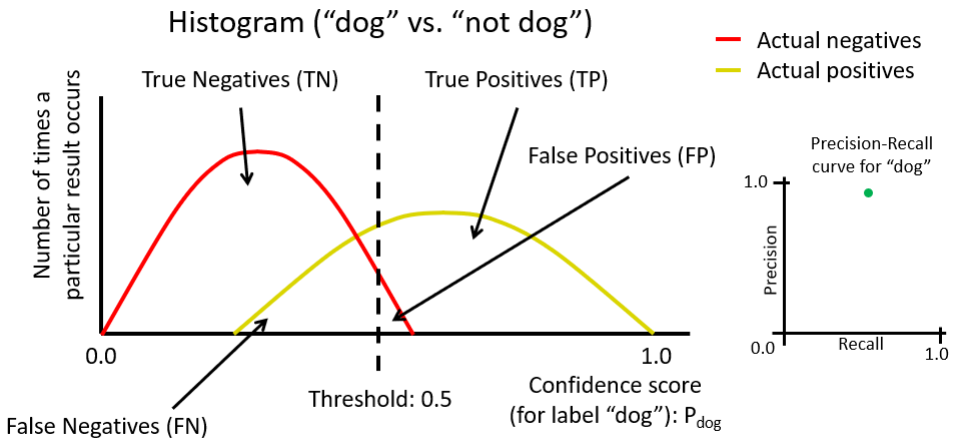
$$Recall = \frac{TP}{TP + FN} = \frac{\square}{\square + \square}$$

$$= \frac{199}{199 + 6} = 0.971$$

Extra

111

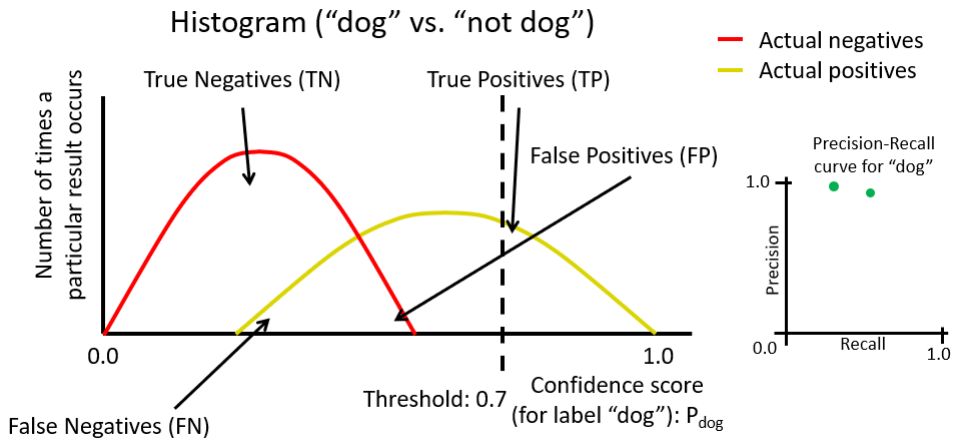
Object detection – Performance



Extra

112

Object detection – Performance



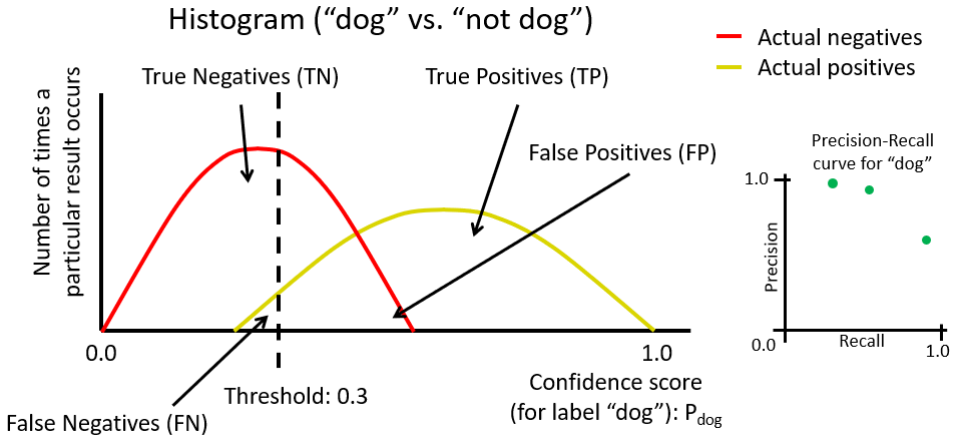
Extra

113

hogeschool vives

113

Object detection – Performance



Extra

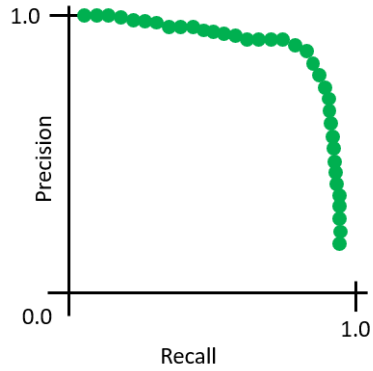
114

hogeschool vives

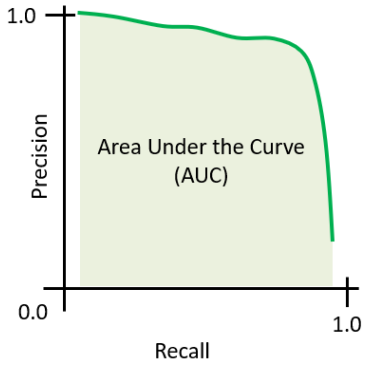
114

Object detection – Performance

Precision-Recall curve for “dog”



Average Precision (AP)



Extra

115

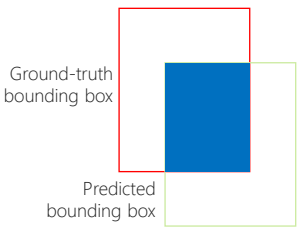
hogeschool vives

115

Object detection – Performance

Intersection over Union (IoU)

Intersection: Area of overlap



Union: Area encompassed by both boxes



$$IoU = \frac{Intersection}{Union} = \frac{area\ of\ overlap}{area\ of\ union}$$

0.0 = complete miss
1.0 = perfect match

Object: IoU >= 0.5

Extra

116

hogeschool vives

116